

Edge Camera AI Control System

RTSP + YOLOv11 + Llama3.2:1B + RAG + LoRA 기반 분산 Edge Camera 관제 플랫폼 기술 기획서

문서 목적: Raspberry Pi 4B 기반 Edge AI Camera Node와 중앙 관제 시스템의 실제 구현 가능한 아키텍처, API, 데이터베이스, 배포 및 운영 구조 정의

대상 독자: 임베디드 AI 개발자, 백엔드 개발자, DevOps 담당자, 관제 시스템 운영자

작성 기준일: 2026-06-08

버전: v1.0

목차

1. 프로젝트 개요
2. 시스템 정의
3. 시스템 목표
4. 요구사항 정의
5. 전체 시스템 아키텍처
6. Edge Camera Node 아키텍처
7. Control Center 아키텍처
8. RTSP 처리 구조
9. YOLOv11 구조
10. Scene Summary 구조
11. Llama3.2:1B 구조
12. RAG 구조
13. LoRA 구조
14. Prompt Engineering 구조
15. Memory 구조
16. Camera Node API
17. Central Control API
18. Authentication 구조
19. Camera Registration 구조
20. Event 처리 구조
21. Alert 처리 구조
22. Detection 결과 저장 구조
23. Scene Summary 저장 구조
24. Database 설계
25. Data Flow
26. Video Streaming Flow
27. Natural Language Query Flow
28. Multi Camera Query Flow
29. Queue 구조
30. Monitoring 구조
31. Logging 구조
32. Security 구조
33. Raspberry Pi 역할
34. GPU Training Server 역할
35. LoRA Training 구조
36. RAG Index 구조
37. Prompt Template
38. JSON Output Format
39. Deployment 구조
40. 설치 절차

- l1. 운영 절차
- l2. 장애 대응 구조
- l3. 성능 고려사항
- l4. 개발 일정
- l5. 테스트 방법
- l6. 확장 구조
- l7. 위험 요소 분석
- l8. 향후 개선 방향
- l9. 최종 결론

1. 프로젝트 개요

Edge Camera AI Control System은 RTSP IP Camera와 Raspberry Pi 4B를 Edge Camera Node로 구성하고, 각 노드에서 객체 탐지, 장면 요약, 자연어 질의 응답, 이벤트 및 알림 생성을 수행하는 분산 관제 플랫폼이다. 중앙 관제 시스템은 여러 Camera Node를 등록, 조회, 제어하며 운영자는 영상, 객체, 이벤트, 알림, 상태, 설정을 통합 화면과 API로 관리한다.

핵심 원칙: 영상 원본은 가능한 Edge에서 처리하고 중앙에는 요약, 탐지 결과, 이벤트, 알림, 최소한의 스냅샷만 전송한다. 이를 통해 네트워크 사용량, 중앙 서버 부하, 개인정보 노출 범위를 줄인다.

2. 시스템 정의

구성 요소	정의
RTSP IP Camera	H.264/H.265 기반 실시간 영상 스트림을 제공하는 네트워크 카메라
Edge Camera Node	Raspberry Pi 4B에서 실행되는 영상 수신, YOLO 추론, LLM 응답, REST API 제공 단위
Control Center	카메라 등록, 질의 라우팅, 이벤트/알림 관리, 대시보드 API를 제공하는 중앙 서버
GPU Training Server	YOLO 학습, LoRA 학습, RAG 인덱스 빌드, 모델 검증을 수행하는 고성능 서버

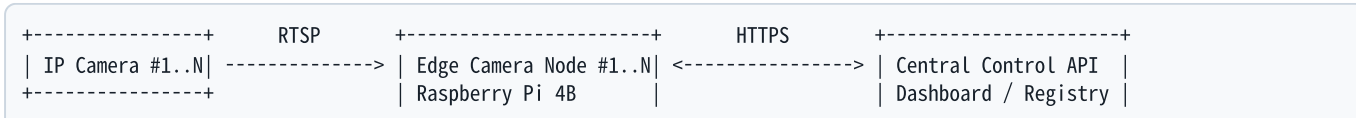
3. 시스템 목표

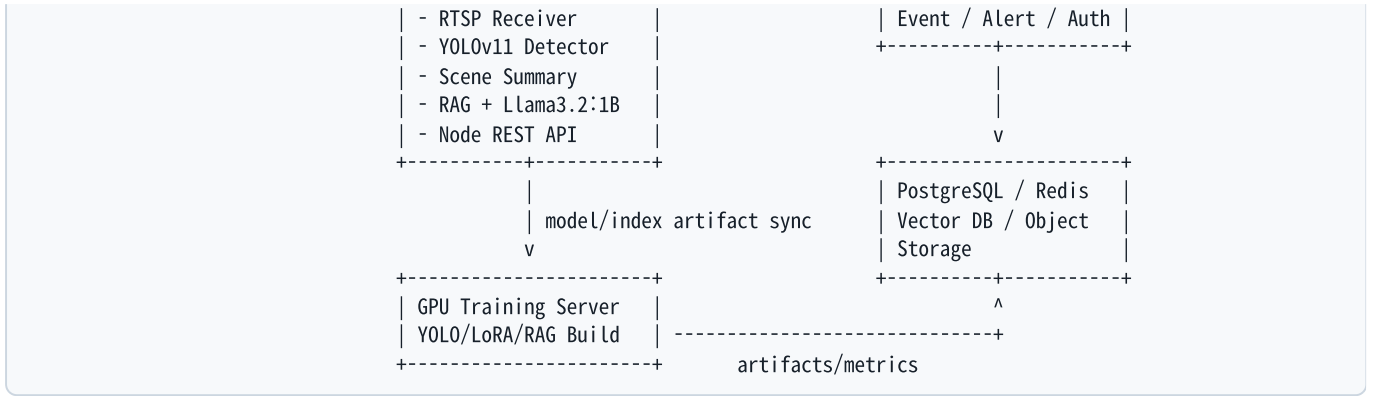
- xN개의 Camera Node를 수평 확장하고 노드별 독립 운영을 지원한다.
- RTSP 수신부터 자연어 응답까지 Edge Node 단에서 1차 처리한다.
- 중앙 관제는 HTTP GET/POST로 Camera Node에 자연어 질의를 전달하고 JSON 응답을 수신한다.
- 탐지 결과, 장면 요약, 이벤트, 알림, 헬스 상태를 데이터베이스에 일관되게 저장한다.
- Raspberry Pi와 GPU Server의 역할을 명확히 분리하여 현장 배포 비용과 학습 성능을 균형 있게 확보한다.

4. 요구사항 정의

분류	요구사항	구현 기준
영상	RTSP 수신, 재연결, 최신 프레임 캐시	OpenCV/GStreamer, ring buffer, watchdog
AI	YOLOv11 객체 탐지, Llama3.2:1B 응답, RAG 검색	Edge 추론은 경량 모델, 학습은 GPU Server
API	Camera Node와 Control Center REST API	FastAPI, JSON, OpenAPI 문서
운영	헬스 모니터링, 로그, 이벤트, 알림	Prometheus exporter, structured logging
보안	인증, 권한, TLS, 토큰 관리	JWT/OAuth2, Node API key, mTLS 선택 적용

5. 전체 시스템 아키텍처





6. Edge Camera Node 아키텍처

Input Layer

RTSP Receiver, frame decoder, frame ring buffer, camera metadata loader

AI Layer

YOLOv11 detector, detection cache, scene summarizer, RAG retriever, Llama inference runtime

Service Layer

FastAPI server, query orchestrator, event generator, alert evaluator, health monitor

Storage Layer

SQLite local DB, JSON cache, optional local vector index, image snapshot directory

Edge Node는 중앙 장애 상황에서도 제한된 독립 동작이 가능해야 한다. 중앙 서버와 연결이 끊긴 경우 이벤트와 알림은 로컬 큐에 적재하고 재연결 후 재전송한다.

7. Control Center 아키텍처

Control Center는 카메라 레지스트리, 질의 라우터, 이벤트/알림 집계, 인증, 대시보드 API, 설정 배포 기능을 담당한다. 영상 자체의 장기 저장은 선택 사항이며, 기본 설계는 Edge Node에서 최신 프레임과 저주기 스냅샷만 제공받는다.

```

Dashboard UI -> Central REST API -> Query Manager -> Camera Node API
| -> Registry Manager -> PostgreSQL
| -> Event Manager -> PostgreSQL / Redis
| -> Alert Manager -> Notification Channel
| -> Config Manager -> Camera Node
  
```

8. RTSP 처리 구조

- RTSP URL, 인증 정보, transport mode, frame skip, target FPS를 설정 파일로 관리한다.
- 수신 스레드는 최신 프레임 ring buffer를 갱신하고, 추론 스레드는 buffer에서 가장 최신 프레임만 소비한다.
- 연결 실패 시 exponential backoff로 재연결하며, 일정 횟수 이상 실패하면 health 상태를 degraded로 전환한다.
- 영상 API는 원본 RTSP를 직접 중계하거나 MJPEG/HLS proxy를 제공한다. 저사양 Pi에서는 MJPEG snapshot 중심을 권장한다.

9. YOLOv11 구조

YOLOv11은 Edge Node에서 객체 탐지용으로 사용한다. Raspberry Pi 4B에서는 입력 해상도, FPS, 모델 크기를 제한하고 필요 시 INT8/NCNN/ONNX Runtime 변환 모델을 사용한다.

항목	권장값
입력 해상도	320x320 또는 416x416부터 시작, 정확도 필요 시 640x640
추론 주기	1~5 FPS, 관제용 요약은 2초 단위 집계
출력	class, confidence, bbox, track_id, timestamp
후처리	NMS, confidence threshold, ROI filter, class allow/deny list

10. Scene Summary 구조

Scene Summary는 최근 탐지 결과, 객체 수 변화, 위치, 센서 데이터, 이전 요약을 결합하여 생성한다. Edge Node는 모든 프레임을 LLM에 전달하지 않고 구조화된 detection cache를 기반으로 요약을 생성한다.

```
{
  "camera_id": "cam_001",
  "window_sec": 30,
  "summary": "창고 입구에 사람 2명과 지게차 1대가 감지됨",
  "objects": {"person": 2, "forklift": 1},
  "risk_level": "medium",
  "updated_at": "2026-06-08T10:15:00+09:00"
}
```

11. Llama3.2:1B 구조

Llama3.2:1B는 Edge Node의 자연어 응답 생성기로 사용한다. 입력은 raw image가 아니라 latest frame metadata, YOLO detection result, detection count/confidence, camera location, sensor data, RAG context, previous scene summary, user question을 조합한 prompt다. 출력은 반드시 JSON으로 제한한다.

- Runtime: Ollama, llama.cpp, 또는 ONNX/gguf runtime 중 Pi 환경에 맞춰 선택
- Context: 최근 요약과 RAG 문서를 짧게 압축하여 2k~4k token 내 유지
- Guardrail: JSON schema validation 실패 시 재시도 prompt 또는 rule-based fallback 사용

12. RAG 구조

RAG는 카메라 위치, 위험 규칙, 작업장 매뉴얼, 금지 구역, 장비 목록, 운영 정책을 검색하여 질의 응답 품질을 높인다. Edge Node는 로컬 vector index를 사용하고, 중앙 서버는 전체 지식 베이스와 인덱스 빌드 파이프라인을 관리한다.

```
User Question + Camera Context
-> query embedding
-> local vector search top-k
-> metadata filter(camera_id, location, domain)
-> prompt context injection
-> JSON answer
```

13. LoRA 구조

LoRA는 Llama3.2:1B의 관제 도메인 응답 형식, 위험 판단 문체, JSON 출력 안정성을 개선하기 위해 사용한다. 학습은 GPU Training Server에서 수행하고 Edge Node에는 병합 또는 adapter 형태의 경량 artifact를 배포한다.

학습 데이터	예시
질의-응답	작업장 위험 질의, 객체 현황 질의, 카메라 상태 질의
JSON 출력	answer, risk_level, detected_objects, recommended_action 필드 고정
부정 예시	근거 없는 객체 추정 금지, 낮은 confidence 경고

14. Prompt Engineering 구조

Prompt는 system, context, memory, question, output schema 블록으로 구성한다. Edge Node는 prompt version을 저장하고 각 응답에 사용된 version을 반환한다.

```
System: You are an edge camera safety assistant. Return JSON only.
Camera: {camera_id}, {location}, {status}
Latest detections: {detections}
Counts: {detection_count}
Sensor: {sensor_data}
Previous summary: {previous_scene_summary}
RAG context: {retrieved_context}
Question: {user_question}
Output schema: {json_schema}
```

15. Memory 구조

- Short-term memory: 최근 30~120초 detection cache, scene summary, alert state
- Session memory: 사용자 질의 이력, 같은 dashboard session의 follow-up query
- Long-term memory: 중앙 DB의 이벤트, 알림, 카메라 설정, RAG 문서
- Eviction: 오래된 frame과 low-confidence detection은 TTL 기반 삭제

16. Camera Node API

Method	Path	설명
GET	/api/v1/health	노드 상태, RTSP 상태, 모델 상태 조회
GET	/api/v1/frame/latest	최신 JPEG frame 또는 metadata 조회
GET	/api/v1/detections/latest	최신 YOLO detection 결과 조회
GET	/api/v1/summary/latest	최신 scene summary 조회
GET	/api/v1/query	query 파라미터 기반 간단 자연어 질의
POST	/api/v1/query	JSON body 기반 자연어 질의
POST	/api/v1/config	threshold, FPS, ROI 등 설정 변경

```
GET /api/v1/query?question=Is%20there%20a%20person%3F

POST /api/v1/query
{
  "request_id": "req_20260608_0001",
  "question": "Camera 1 describe current scene",
  "include_frame": false,
  "max_context_docs": 3
}
```

17. Central Control API

Method	Path	설명
POST	/api/v1/cameras/register	Camera Node 등록
GET	/api/v1/cameras	카메라 목록 조회
POST	/api/v1/query	단일 카메라 자연어 질의
POST	/api/v1/query/multi	다중 카메라 질의
GET	/api/v1/events	이벤트 조회
GET	/api/v1/alerts	알림 조회
POST	/api/v1/cameras/{camera_id}/config	카메라 설정 변경

```
POST /api/v1/query
{
  "camera_id": "cam_003",
  "question": "Is there a person in Camera 3?"
}
```


18. Authentication 구조

운영자 인증은 중앙 서버에서 JWT 기반으로 처리한다. Camera Node와 중앙 서버 간 통신은 Node API key 또는 mTLS 를 사용한다. API key는 등록 시 발급하고 주기적으로 회전한다.

- Dashboard user: OAuth2 password/client credentials 또는 사내 SSO 연동
- Central to Node: Authorization: Bearer <node_access_token>
- Node to Central: registration token, heartbeat token, event push token 분리
- 권한: viewer, operator, admin, service-account

19. Camera Registration 구조

1. 관리자가 Control Center에서 camera_id, location, rtsp_url, node_endpoint를 등록
2. Central Server가 registration_token 발급
3. Edge Node가 /api/v1/cameras/register로 node metadata 전송
4. Central Server가 node_access_token 발급
5. Edge Node가 heartbeat와 event push 시작

20. Event 처리 구조

Event는 탐지 결과나 상태 변화가 의미 있는 시점에 생성된다. 예: person detected, restricted zone entry, camera offline, model degraded, repeated low confidence.

필드	설명
event_type	detection, scene_change, health, config, security
severity	info, low, medium, high, critical
evidence	detection ids, snapshot path, summary id, confidence
delivery	local queue 저장 후 central push, 실패 시 retry

21. Alert 처리 구조

Alert는 Event 중 운영 조치가 필요한 항목이다. Alert rule은 중앙에서 관리하고 Edge Node에 동기화한다.

```
{
  "rule_id": "warehouse_person_after_hours",
  "condition": "object.person.count >= 1 AND time between 22:00 and 06:00",
  "severity": "high",
  "action": ["dashboard_popup", "webhook", "sms"]
}
```

22. Detection 결과 저장 구조

Edge Node는 최근 detection을 로컬 SQLite와 메모리 cache에 저장한다. 중앙 서버는 이벤트로 승격된 detection 또는 주기 샘플만 저장하여 저장량을 제어한다.

```
{
  "detection_id": "det_001",
```

```
"camera_id": "cam_001",
"class_name": "person",
"confidence": 0.91,
"bbox": [120, 80, 260, 420],
"frame_ts": "2026-06-08T10:15:01+09:00"
}
```

23. Scene Summary 저장 구조

Scene Summary는 시간 window 단위로 저장하며, query 응답과 event 판단의 근거가 된다. 동일 장면이 지속되면 요약 갱신하지 않고 last_seen_at만 갱신해 중복 저장을 줄인다.

24. Database 설계

PostgreSQL Central Schema

```
CREATE TABLE cameras (
  camera_id VARCHAR(64) PRIMARY KEY,
  name VARCHAR(128) NOT NULL,
  location TEXT NOT NULL,
  node_endpoint TEXT NOT NULL,
  rtsp_url_encrypted TEXT,
  status VARCHAR(32) NOT NULL DEFAULT 'unknown',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE detections (
  detection_id UUID PRIMARY KEY,
  camera_id VARCHAR(64) REFERENCES cameras(camera_id),
  class_name VARCHAR(64) NOT NULL,
  confidence NUMERIC(5,4) NOT NULL,
  bbox JSONB NOT NULL,
  frame_ts TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE scene_summaries (
  summary_id UUID PRIMARY KEY,
  camera_id VARCHAR(64) REFERENCES cameras(camera_id),
  summary TEXT NOT NULL,
  object_counts JSONB NOT NULL,
  risk_level VARCHAR(16) NOT NULL,
  window_start TIMESTAMPTZ NOT NULL,
  window_end TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE events (
  event_id UUID PRIMARY KEY,
  camera_id VARCHAR(64) REFERENCES cameras(camera_id),
  event_type VARCHAR(64) NOT NULL,
  severity VARCHAR(16) NOT NULL,
```

```
payload JSONB NOT NULL,  
occurred_at TIMESTAMPTZ NOT NULL,  
created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE alerts (  
  alert_id UUID PRIMARY KEY,  
  event_id UUID REFERENCES events(event_id),  
  camera_id VARCHAR(64) REFERENCES cameras(camera_id),  
  status VARCHAR(32) NOT NULL DEFAULT 'open',  
  severity VARCHAR(16) NOT NULL,  
  message TEXT NOT NULL,  
  acknowledged_by VARCHAR(128),  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  closed_at TIMESTAMPTZ  
);
```

Edge Local SQLite Schema

```
CREATE TABLE local_detections (  
  id TEXT PRIMARY KEY,  
  class_name TEXT NOT NULL,  
  confidence REAL NOT NULL,  
  bbox TEXT NOT NULL,  
  frame_ts TEXT NOT NULL  
);  
  
CREATE TABLE local_queue (  
  id TEXT PRIMARY KEY,  
  topic TEXT NOT NULL,  
  payload TEXT NOT NULL,  
  retry_count INTEGER NOT NULL DEFAULT 0,  
  created_at TEXT NOT NULL  
);
```

25. Data Flow

```
RTSP Frame -> Frame Buffer -> YOLO Detector -> Detection Cache
                                         | -> Event Rule Engine -> Local Queue -> Central Event API
Detection Cache + Sensor + RAG + Memory -> Scene Summary -> Local DB -> Central Summary API
User Question -> Central Query API -> Node Query API -> Llama JSON Answer -> Dashboard
```

26. Video Streaming Flow

1. Dashboard가 Central API에서 camera endpoint와 권한을 확인한다.
2. 영상 확인 요청은 중앙 proxy 또는 Edge Node direct proxy로 전달된다.
3. Edge Node는 RTSP 최신 프레임을 JPEG snapshot, MJPEG stream, 또는 HLS segment로 제공한다.
4. 저대역 환경에서는 1 FPS snapshot polling을 기본값으로 사용한다.

27. Natural Language Query Flow

```
Operator
-> POST /api/v1/query on Central
-> Camera Registry lookup
-> POST /api/v1/query on Edge Node
-> collect latest frame metadata, YOLO detections, counts, confidence, location, sensor, RAG, previous summary
-> build prompt
-> Llama3.2:1B inference
-> JSON schema validation
-> response stored and returned
```

질의 예시:

- "Camera 1 describe current scene"
- "Is there a person in Camera 3?"
- "Any dangerous situation in warehouse?"
- "What objects are detected?"
- "Summarize current activity"

28. Multi Camera Query Flow

다중 카메라 질의는 중앙 서버가 수행한다. 질의의 location 또는 camera group을 해석하고 여러 Edge Node에 병렬 요청한 뒤 결과를 aggregation prompt 또는 rule-based merge로 통합한다.

```
POST /api/v1/query/multi
{
  "scope": {"location": "warehouse"},
  "question": "Any dangerous situation in warehouse?",
  "timeout_ms": 8000
}
```

29. Queue 구조

Queue	위치	용도
frame_queue	Edge memory	최신 프레임 ring buffer
detection_queue	Edge memory	YOLO 후처리와 이벤트 평가 분리
event_outbox	Edge SQLite	중앙 전송 실패 이벤트 재시도
alert_queue	Central Redis	알림 전송 비동기 처리

30. Monitoring 구조

- Edge metrics: CPU, memory, temperature, disk, RTSP connected, inference latency, FPS, queue backlog
- Central metrics: API latency, node heartbeat age, DB connection, alert delivery status
- 수집: Prometheus node exporter 또는 FastAPI metrics endpoint
- 시각화: Grafana dashboard, Control Center health page

31. Logging 구조

모든 서비스는 JSON structured log를 남긴다. request_id, camera_id, node_id, event_id, latency_ms, model_version, prompt_version을 공통 필드로 사용한다.

```
{
  "level": "INFO",
  "service": "edge-node",
  "camera_id": "cam_001",
  "request_id": "req_001",
  "message": "query completed",
  "latency_ms": 1320
}
```

32. Security 구조

- RTSP credential은 평문 저장 금지, 중앙 DB에는 암호화 저장
- Edge Node API는 내부망 접근을 기본으로 하고 외부 노출 시 TLS와 token 필수
- 프레임 snapshot 저장 시 보존 기간과 마스킹 정책 적용
- LLM 응답에는 내부 토큰, RTSP URL, 시스템 prompt를 포함하지 않도록 출력 필터 적용
- 관리 API는 admin 권한과 audit log를 요구한다.

33. Raspberry Pi 역할

수행	비고
RTSP 수신, 최신 프레임 cache	현장 카메라와 같은 네트워크 권장
YOLOv11 경량 추론	저해상도, 낮은 FPS, quantized model 권장
Detection cache, scene summary	로컬 SQLite/메모리 기반
RAG 검색과 Llama3.2:1B 응답	짧은 context와 JSON 출력 제한
Node REST API, health monitor	systemd 서비스로 자동 시작

34. GPU Training Server 역할

수행	비고
YOLO 데이터셋 학습/검증	현장 객체 클래스 추가 학습
Llama LoRA fine-tuning	질의응답, JSON 출력, 위험 판단 데이터셋
RAG embedding/index 생성	문서 chunking, vector index build
모델 변환과 배포 artifact 생성	ONNX/NCNN/GGUF 등 Edge 친화 포맷
성능 벤치마크	정확도, latency, memory, thermal 측정

35. LoRA Training 구조

```
Collect domain Q/A -> Clean JSON schema -> Split train/valid
-> Fine-tune Llama3.2:1B with LoRA
-> Evaluate JSON validity and task accuracy
-> Export adapter or merged model
-> Deploy to Edge Nodes
```

평가 지표는 JSON validity, hallucination rate, risk classification accuracy, average latency, memory footprint로 구성한다.

36. RAG Index 구조

- 문서: 작업장 안전 규정, 카메라 위치 설명, 장비 목록, 알림 롤 설명, 운영 매뉴얼
- Chunk: 300~600 tokens, overlap 50~100 tokens
- Metadata: camera_id, location, document_type, version, effective_date
- Index: Edge 로컬 FAISS/Chroma compatible index, 중앙 원본 index registry

37. Prompt Template

```
You are Edge Camera AI Control System.
Rules:
- Return valid JSON only.
- Do not invent objects that are not in detections.
```

- If confidence is low, state uncertainty.
- Use RAG context only when relevant.

Input:

```
camera_id: {{camera_id}}
location: {{location}}
latest_frame_time: {{frame_time}}
detections: {{detections_json}}
detection_count: {{count_json}}
sensor_data: {{sensor_json}}
previous_scene_summary: {{summary}}
rag_context: {{rag_context}}
question: {{question}}
```

Required JSON schema:

```
{{schema}}
```

38. JSON Output Format

```
{
  "request_id": "req_20260608_0001",
  "camera_id": "cam_001",
  "answer": "현재 창고 입구에 사람 2명이 감지되었습니다.",
  "scene_summary": "사람 2명이 입구 부근에 있으며 위험 장비는 감지되지 않았습니다.",
  "detected_objects": [
    {"class_name": "person", "count": 2, "max_confidence": 0.93}
  ],
  "risk_level": "low",
  "recommended_action": "모니터링을 유지하십시오.",
  "evidence": {
    "frame_time": "2026-06-08T10:15:01+09:00",
    "summary_id": "sum_001",
    "rag_docs": ["warehouse_policy_v3"]
  },
  "model": {
    "name": "llama3.2:1b",
    "prompt_version": "edge_qa_v1"
  }
}
```

39. Deployment 구조

```
[Raspberry Pi Edge Node]
/opt/edge-camera-node
  app/          FastAPI, RTSP, YOLO, RAG, LLM modules
  models/       yolo model, llama gguf/adapter
  rag_index/    local vector index
  data/         sqlite, snapshots, local queue
  config/node.yaml
  systemd/edge-camera-node.service

[Central Control Server]
/opt/edge-control-center
  app/          FastAPI central backend
  web/          dashboard frontend
  migrations/   PostgreSQL schema
  config/server.yaml
  docker-compose.yml  postgres, redis, api, dashboard

[GPU Training Server]
/opt/edge-ai-training
  datasets/
  training/yolo/
  training/lora/
  rag_builder/
  artifacts/
```

40. 설치 절차

Raspberry Pi

```
sudo apt update
sudo apt install -y python3-venv python3-opencv ffmpeg
python3 -m venv .venv
. .venv/bin/activate
pip install -r requirements-edge.txt
cp config/node.example.yaml config/node.yaml
sudo cp systemd/edge-camera-node.service /etc/systemd/system/
sudo systemctl enable --now edge-camera-node
```

Central Server

```
docker compose up -d postgres redis
python3 -m venv .venv
. .venv/bin/activate
pip install -r requirements-central.txt
alembic upgrade head
uvicorn app.main:app --host 0.0.0.0 --port 8080
```

41. 운영 절차

1. Central Control Center에 카메라와 위치를 등록한다.
2. Edge Node 설정 파일에 camera_id, RTSP URL, 중앙 endpoint, token을 입력한다.
3. 노드 서비스를 시작하고 heartbeat 상태를 확인한다.
4. Dashboard에서 최신 프레임, detection, summary, event, alert를 확인한다.

5. 운영 중 threshold, ROI, alert rule은 중앙에서 변경하고 노드에 동기화한다.

42. 장애 대응 구조

장애	탐지	대응
RTSP 끊김	frame age 초과	재연결, camera offline event, 운영자 alert
Pi 과열	temperature threshold	FPS 감소, 모델 추론 주기 증가, 냉각 점검 alert
LLM JSON 실패	schema validation error	재시도, fallback JSON 생성, 로그 저장
Central 연결 실패	heartbeat push 실패	local queue 적재, backoff retry
DB 장애	health check 실패	Redis 임시 큐, read-only mode, 관리자 알림

43. 성능 고려사항

- Raspberry Pi 4B는 CPU, 메모리, 발열 제약이 크므로 YOLO와 LLM 동시 실행 시 추론 주기를 분리한다.
- LLM 질의는 사용자가 요청할 때 실행하고, 주기 요약은 짧은 prompt와 낮은 빈도로 수행한다.
- 탐지 confidence threshold와 ROI를 조정하여 불필요한 event를 줄인다.
- Multi camera query는 중앙 서버에서 병렬 요청하되 timeout과 partial result를 지원한다.
- 스냅샷 저장은 retention 정책을 두어 SD 카드 수명과 저장 공간을 보호한다.

44. 개발 일정

단계	기간	산출물
1. PoC	2주	RTSP 수신, YOLO 추론, Node API
2. Edge AI 통합	3주	Detection cache, summary, Llama JSON 응답
3. Central 구축	3주	Registry, query routing, event/alert API, DB
4. RAG/LoRA	3주	RAG index, LoRA 학습, artifact 배포
5. 운영 안정화	2주	모니터링, 보안, 장애 대응, 문서화

45. 테스트 방법

- Unit test: detection parser, prompt builder, JSON validator, alert rule evaluator
- Integration test: Central query API에서 Edge Node mock 또는 실제 Pi 호출
- Load test: xN Camera Node heartbeat, event push, query burst
- AI test: 객체 탐지 정확도, scene summary 일관성, JSON validity, 위험 판단 정확도
- Failure test: RTSP disconnect, central unavailable, DB unavailable, model load failure

```
curl -X POST http://central:8080/api/v1/query \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer ${TOKEN}" \  
-d '{"camera_id":"cam_001","question":"What objects are detected?"}'
```

46. 확장 구조

Camera Node는 독립 REST endpoint와 heartbeat를 가지므로 xN개로 확장 가능하다. 중앙 서버는 camera group, location, tag 기반으로 node를 검색하고 질의를 분산한다. 대규모 환경에서는 API gateway, message broker, read replica, object storage, vector DB cluster를 추가한다.

47. 위험 요소 분석

위험	영향	완화
Pi 성능 부족	낮은 FPS, 응답 지연	모델 경량화, 추론 주기 조정, Edge TPU/NPU 옵션
LLM hallucination	잘못된 관제 판단	JSON schema, detection 근거 제한, confidence 표시
RTSP 네트워크 불안정	영상 손실	재연결, 상태 알림, frame age 모니터링
개인정보 이슈	법적/운영 리스크	Edge 처리, 저장 최소화, 접근 제어, retention
알림 과다	운영 피로	룰 튜닝, 중복 억제, severity 정책

48. 향후 개선 방향

- 객체 추적 기반 행동 인식과 위험 행동 탐지 추가
- 멀티모달 VLM 적용을 통한 frame 직접 설명 기능 강화
- Edge TPU, Hailo, Jetson 등 가속 하드웨어별 모델 배포 profile 지원
- Federated learning 또는 privacy-preserving analytics 도입
- 대시보드에서 prompt, RAG 문서, alert rule을 버전 관리하는 운영 도구 추가

49. 최종 결론

본 기획서는 RTSP IP Camera와 Raspberry Pi 4B를 기반으로 분산 Edge Camera AI Control System을 구축하기 위한 실제 구현 수준의 구조를 정의했다. Edge Node는 영상 수신, YOLOv11 탐지, Scene Summary, RAG 검색, Llama3.2:1B JSON 응답, 이벤트/알림, 헬스 모니터링을 담당하고, Control Center는 카메라 등록, 질의 라우팅, 상태/이벤트/알림 통합 관제를 담당한다. GPU Training Server는 YOLO, LoRA, RAG artifact를 생성하여 Edge Node에 배포한다.

이 구조는 중앙 서버 의존도를 낮추면서도 xN개 Camera Node 확장, 자연어 질의, 운영 안정성, 보안 정책을 함께 만족하도록 설계되었다. PoC 단계에서는 RTSP 수신, YOLO 탐지, Node Query API, Central Query Routing을 우선 구현하고 이후 RAG, LoRA, 운영 자동화를 순차적으로 확장하는 접근이 적합하다.